

DataShare

Plateforme de transfert securise de fichiers, prototype MVP.

Developee avec un copilote IA cantonne a un perimetre restreint et supervise.

Projet	DataShare : MVP OpenClassrooms
Auteur	Omeima Lassakeur
Version	1.0.0
Date	Mai 2026
Stack	React 19 / TypeScript . Spring Boot 3 . PostgreSQL 16
Repository	github.com/astriiiixxx/Projet3

Sommaire

1. Architecture de l'application
2. Choix technologiques justifiés
3. Modele de donnees
4. Documentation d'API
5. Securite et gestion des acces
6. Qualite, tests et maintenance
7. Processus d'installation et d'execution
8. Utilisation de l'IA dans le developpement

1. Architecture de l'application

DataShare suit une architecture full-stack trois-tiers, totalement stateless cote serveur. Le frontend React communique avec le backend Spring Boot via une API REST/JSON. Le backend persiste les metadonnees dans PostgreSQL et stocke les binaires sur le systeme de fichiers local. L'authentification utilise un JWT signe HMAC-SHA256, ce qui permet une mise a l'echelle horizontale sans replication de session.

Diagramme d'architecture globale

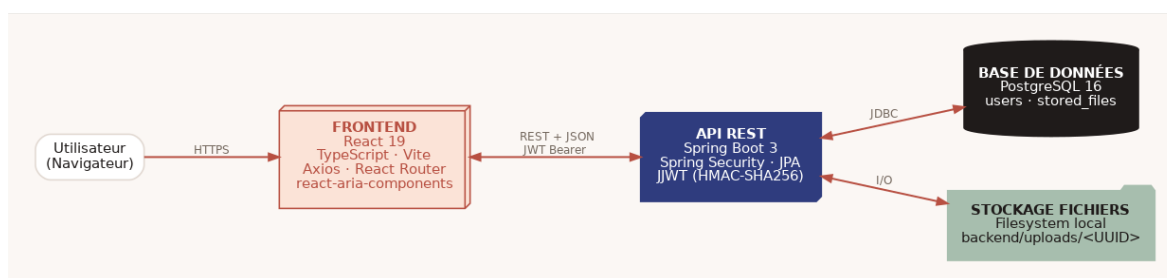


Figure 1 : Vue trois-tiers de DataShare — client React, API Spring Boot, PostgreSQL pour les metadonnees et filesystem local pour les binaires.

Architecture en couches du backend

L'API suit une architecture en couches strictes, choix classique de l'écosystème Spring qui separe clairement les responsabilites : la couche Security filtre les requetes entrantes, les Controllers exposent les endpoints REST, les Services portent la logique metier, les Repositories font l'accès BDD via Spring Data JPA, et le stockage physique est delegue a FileStorageService. Cette separation rend chaque couche testable independamment.

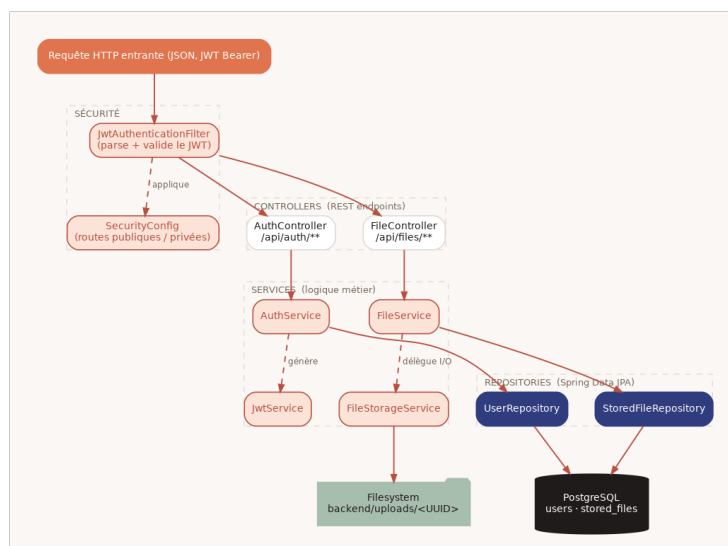


Figure 2 : Cheminement d'une requete HTTP : filtre JWT > controleur REST > service metier > repository JPA > PostgreSQL ou filesystem.

Briques principales

- **Frontend** : SPA React (Vite). Pages Home, Login, Register, Upload, Download, MySpace, UiPreview et un design system maison (AppButton, AppTextField, AppCallout, AppHeader, AppSelect, AppSegmentedControl).
- **Backend** : API REST Spring Boot. Controleurs AuthController, FileController. Services AuthService, FileService, FileStorageService, JwtService.
- **Base de donnees** : PostgreSQL 16. Deux tables users et stored_files, en relation 1..n optionnelle.

- **Stockage fichiers** : systeme de fichiers local sous backend/uploads/. Chaque fichier renommé en UUID.
- **Securite** : Spring Security stateless ; JwtAuthenticationFilter durci pour ignorer les routes publiques.

2. Choix technologiques justifies

Trois criteres ont guide les choix : (1) compatibilite avec la stack imposee par les specifications, (2) maturite de l'ecosysteme pour livrer un MVP avec un developpeur unique epaulee par un copilote IA, (3) facilite d'evolution post-MVP.

Element	Technologie choisie	Alternatives	Justification
Frontend	React 19 + TypeScript	Angular, Vue.js	Ecosysteme mature, typage fort, courbe douce, composants accessibles.
Build	Vite 7	Webpack, Parcel	HMR tres rapide, build optimise, configuration minimale.
Routage	React Router 7	TanStack Router	Standard de fait, support natif des routes imbriquees (Outlet).
HTTP client	Axios	fetch natif	Intercepteurs : injection automatique du JWT et redirection sur 401.
UI	react-aria-components	Headless UI, MUI	Accessibilite native (focus, ARIA, clavier), composants non opiniatres.
Backend	Spring Boot 3 + Java 21	NestJS, .NET, Symfony	Le plus robuste de la liste imposee. Spring Security mature.
Securite	Spring Security + JWT	Auth0, Keycloak	Pas de dependance externe pour le MVP. BCrypt natif, JWT HMAC-SHA256.
ORM	Spring Data JPA	JOOQ, MyBatis	Repositories types, requetes derivees des noms de methode.
Base de donnees	PostgreSQL 16	MongoDB	Donnees fortement relationnelles, transactions ACID.
Stockage	Filesystem local	AWS S3, MinIO	Suffisant pour MVP. Migration vers S3 prevue v1.2 (PERF.md).
Tests backend	JUnit 5 + Mockito	TestNG	Standard de l'ecosysteme Spring.
Tests frontend	Vitest + Testing Lib.	Jest	Compatible Vite, API proche de Jest, plus rapide.

3. Modele de donnees

Le modele est volontairement minimal pour le MVP : deux entites et une relation 1..n. Il couvre US01-US06 et les options US07, US09, US10. La gestion des tags (US08) a ete ecartee du scope pour garantir la qualite sur les options securite.

Note d'evolution : la conception initiale prevoyait une troisieme entite *ShareLink* separee de *FileResource*. Pendant l'implementation, j'ai fusionne les deux car le MVP n'avait besoin que d'un lien par fichier : moins de tables, moins de jointures, plus simple a tester. *ShareLink* reste pertinent en evolution si on veut plusieurs liens par fichier.

Diagramme entite/association

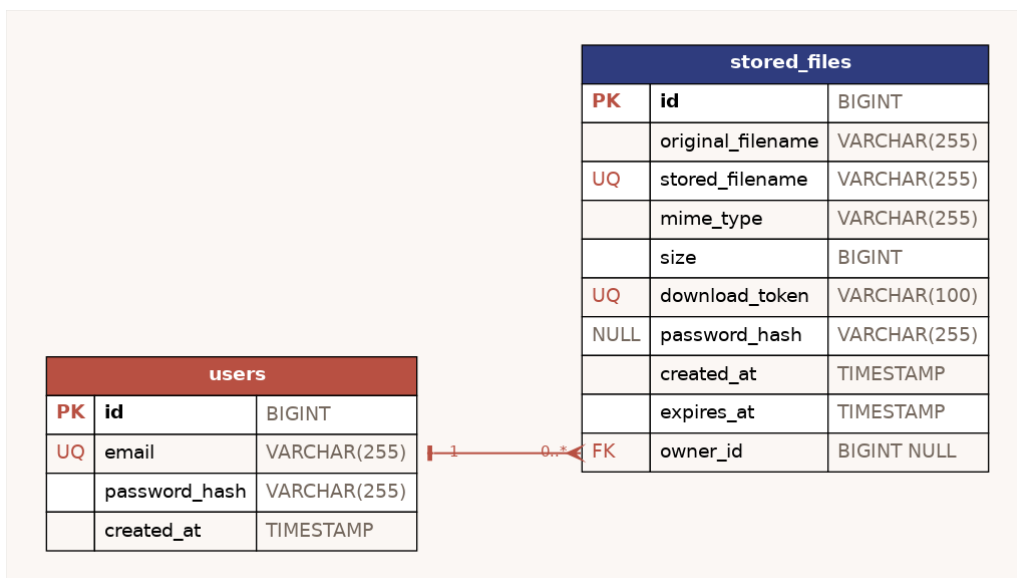


Figure 3 : Modele ER : un utilisateur possede 0 a n fichiers, un fichier appartient a 0 ou 1 utilisateur. owner_id est NULL pour les uploads anonymes (US07).

Table users

Colonne	Type	Contraintes	Description
id	BIGINT	PK, identity	Identifiant interne.
email	VARCHAR(255)	UNIQUE, NOT NULL	Identifiant fonctionnel + login (US03).
password_hash	VARCHAR(255)	NOT NULL	Hash BCrypt (10 rounds).
created_at	TIMESTAMP	NOT NULL	Date d'inscription.

Table stored_files

Colonne	Type	Contraintes	Description
id	BIGINT	PK, identity	Identifiant interne.
original_filename	VARCHAR(255)	NOT NULL	Nom d'origine (affichage uniquement).
stored_filename	VARCHAR(255)	UNIQUE, NOT NULL	UUID utilise sur disque.
mime_type	VARCHAR(255)	NULL	Type MIME detecte.

Colonne	Type	Contraintes	Description
size	BIGINT	NOT NULL	Taille en octets.
download_token	VARCHAR(100)	UNIQUE, NOT NULL	UUID v4 non predictable (US02).
password_hash	VARCHAR(255)	NULL	BCrypt si fichier protege (US09).
created_at	TIMESTAMP	NOT NULL	Date d'envoi.
expires_at	TIMESTAMP	NOT NULL	Date d'expiration (US10, max 7 j).
owner_id	BIGINT	FK users(id), NULL	Proprietaire (NULL si anonyme : US07).

Migration appliquee pendant l'implementation de l'US07 : owner_id rendu nullable via ALTER TABLE stored_files ALTER COLUMN owner_id DROP NOT NULL.

4. Documentation d'API

L'API REST est prefixee par /api. Toutes les reponses sont JSON. L'authentification utilise un header **Authorization: Bearer <jwt>** sur les routes protegees. Les erreurs respectent un format unique ApiResponse (timestamp, status, error, message, path). Une specification OpenAPI 3 sera generee via springdoc-openapi en evolution v1.1.

Liste des endpoints

Methode	Route	Auth	Description
POST	/api/auth/register	Public	Creation de compte (US03).
POST	/api/auth/login	Public	Authentification + generation JWT (US04).
POST	/api/files	JWT	Upload authentifie (US01).
POST	/api/files/anonymous	Public	Upload anonyme (US07).
GET	/api/files	JWT	Historique de l'utilisateur courant (US05).
DELETE	/api/files/{id}	JWT	Suppression d'un fichier proprietaire (US06).
GET	/api/files/public/{token}	Public	Metadonnees publiques avant download (US02).
GET	/api/files/download/{token}	Public	Telechargement binaire. Header X-File-Password si protege (US02+US09).

Exemples concrets

POST /api/auth/login

```
{ "email": "user@example.com", "password": "secret123" }
```

```
Response 200 : { "token": "eyJhbGciOiJIUzI1NiJ9...", "user": { "id": 12, "email": "..."} }
```

Erreurs : 401 (identifiants invalides), 400 (validation).

GET /api/files/download/{token}

X-File-Password: motdepasse # header optionnel, uniquement si fichier protege

200 OK : flux binaire / 401 : mot de passe manquant ou invalide / 404 : token inconnu / 410 : lien expire. Le mot de passe passe en header (jamais en query string) pour ne pas l'exposer dans les logs.

5. Securite et gestion des acces

Authentification

- **Creation de compte (US03)** : email unique en BDD, mot de passe ≥ 8 caracteres, hash BCrypt via PasswordEncoder.
- **Connexion (US04)** : email + mot de passe -> verification BCrypt -> JWT signe HMAC-SHA256, duree 24 h. Secret dans JWT_SECRET.
- **Filtre serveur** : JwtAuthenticationFilter verifie signature, sujet et expiration a chaque requete.
- **Filet client** : jwtUtils.isJwtExpiredOrInvalid teste l'expiration avant chaque envoi. Intercepteur Axios purge localStorage sur 401.

Bug reel rencontre et corrige

Au debut de la phase auth, register et login renvoyaient un 500 Internal Server Error depuis le frontend, alors que les memes appels passaient en Postman. Cause : le JWT envoye automatiquement par l'intercepteur Axios contenait un ancien token expire du localStorage. Le filtre serveur tentait de le parser et levait ExpiredJwtException avant meme d'atteindre le controleur.

- **Cote frontend** : exclusion explicite de /auth/login et /auth/register de l'injection du token.
- **Cote backend** : JwtAuthenticationFilter durci : routes publiques ignorees, token invalide ne fait pas crasher.
- **Cote ops** : nettoyage du localStorage stale via deconnexion + redirection automatique sur 401.

Gestion des roles et permissions

- Pas de role particulier dans le MVP : distinction binaire anonyme / authentifie.
- Routes publiques explicites dans SecurityConfig : /api/auth/**, /api/files/anonymous, /api/files/public/**, /api/files/download/**.
- Controle propriétaire cote serveur sur la suppression : FileService.deleteMyFile leve AccessDeniedException (HTTP 403).

Mesures de securisation

- Mots de passe utilisateur et fichier toujours haches (BCrypt), jamais stockes en clair.
- HTTPS attendu en production (termine au reverse-proxy).
- Validation des entrees cote client ET serveur (taille, type, mot de passe ≥ 6 caracteres).
- Tokens de telechargement : UUID v4 aleatoires, non incrementaux.
- CORS active et restreint a l'origine du frontend.
- CSRF desactive (API stateless avec JWT en header).
- XSS : React echappe par default, aucun dangerouslySetInnerHTML.
- Injection SQL : Spring Data JPA avec parametres prepares.
- Header Content-Disposition: attachment au download pour empecher l'execution navigateur.
- Mot de passe fichier transmis dans un header X-File-Password, jamais en query string.

Limites et protections d'upload

- Taille max : app.file.max-size-bytes (10 Mo par default, ≤ 1 Go par les specs).
- Duree d'expiration max : app.file.max-expiration-days = 7.
- Liste noire d'extensions : .exe, .bat, .cmd, .sh, .msi, .js.

- Stockage sous nom UUID : le nom original n'est conserve qu'en BDD pour l'affichage.

6. Qualite, tests et maintenance

Le detail complet est dans quatre fichiers a la racine du repo.

TESTING.md : plan de tests

- **Unitaires backend** (JUnit 5 + Mockito) : JwtServiceTest, AuthServiceTest, FileServiceTest.
- **Integration backend** (MockMvc standalone) : FileControllerTest.
- **Unitaires frontend** (Vitest + Testing Library) : jwtUtils, AuthContext, intercepteur 401, UploadForm, DownloadPage, MySpacePage.
- **E2E Cypress prévu** : upload anonyme + lien + mot de passe ; cycle complet inscription / login / upload / suppression.
- **Criteres** : couverture ≥ 70 % frontend, ≥ 80 % backend services. Tests rouges = pas de merge.

SECURITY.md

- npm audit cote front, mvn dependency-check:check cote back.
- Decisions documentees : BCrypt, JWT stateless, rotation du secret.

PERF.md

- Test de charge sur GET /api/files/public/{token} avec k6.
- Cible MVP : TTFB ≤ 250 ms (p95), Lighthouse ≥ 85 .
- Budget bundle : ≤ 250 ko gzipped.

MAINTENANCE.md

- SemVer, Conventional Commits.
- Mise a jour mensuelle des dependances avec audit.
- Sauvegards : dump PostgreSQL quotidien, sync uploads/ via rsync.
- Alertes sur 5xx > 1 % et latence p95.

7. Processus d'installation et d'execution

Le detail complet est dans le README.md.

Prerequis

- Node.js 22+ et npm 10+
- Java 21+ et Maven 3.9+
- PostgreSQL 16
- Git 2.40+

Variables d'environnement

- **DB_PASSWORD** : mot de passe du compte PostgreSQL.
- **JWT_SECRET** : cle HMAC-SHA256 base64 (>= 256 bits). Generer : openssl rand -base64 64.

Commandes

```
# 1. Base de donnees
psql -h localhost -U postgres -d postgres
CREATE DATABASE projet3;

# 2. Backend
cd backend
export DB_PASSWORD=<mot_de_passe>
export JWT_SECRET=$(openssl rand -base64 64)
mvn spring-boot:run # API : http://localhost:8000

# 3. Frontend
cd frontend && npm install && npm run dev
# App : http://localhost:5173

# 4. Tests
cd backend && mvn test
cd frontend && npm run test:run
```

8. Utilisation de l'IA dans le developpement

Posture adoptee

L'IA (Claude principalement, ChatGPT en complement) a ete consideree comme un developpeur junior a qui on confie des taches precises et dont on verifie et corrige systematiquement la production. Le pilote garde la maitrise totale de l'architecture, du modele de donnees, de la securite et de la logique metier.

Perimetre confie a l'IA

- **Integration UI react-aria-components** : generation des composants accessibles a partir des maquettes Figma. Validation visuelle sur /ui-preview.
- **Tests** : generation de cas JUnit et Vitest. Adaptation manuelle aux contrats reels du DOM et de l'API.
- **Documentation de base** : premiere redaction README, doc technique, plans MD. Refonte humaine ensuite.

Hors perimetre : decisions humaines

- **Architecture** : choix stack, diagrammes, separation en couches.
- **Modele de donnees** : conception des deux entites, fusion volontaire ShareLink + FileResource en StoredFile.
- **Securite** : flux JWT, routes publiques/privées, strategie X-File-Password, SecurityConfig.
- **Logique metier** : regles d'expiration, validation, controle propriétaire, GlobalExceptionHandler.
- **Diagnostic des bugs** : 500 sur /auth/login (token stale), Content-Type multipart : humain de bout en bout.

Corrections effectuees sur les sorties IA

- **API imaginaires** : endpoints inexistants : refus, reformulation avec le code reel en contexte.
- **Hallucination de getter** : getMaxFileSizeBytes() au lieu du vrai getMaxSizeBytes() : verifie dans le code source.
- **Selecteurs de tests trop generiques** : remplaces par des data-testid precis.
- **Sur-architectures ecartees** : Flyway, Redis, microservices proposes des le MVP : tous refuses.
- Aucun secret reel partage avec les modeles (uniquement des placeholders).

Bilan

L'IA a permis d'accelerer la partie UI accessible, les tests et la premiere redaction documentaire. Sur tout le reste : architecture, modele, securite, metier, debug : la responsabilite est restee integralement humaine. C'est un outil d'augmentation ciblee, pas de substitution.